Handling Timeouts in Playwright

Complete beginner notes with code walkthroughs, DOM examples, realistic scenarios, debugging strategy, and best practices

Based on the supplied "Handling Timeouts" class notes, expanded with practical explanations and current Playwright documentation. Code examples use JavaScript with @playwright/test.

THE CORE IDEA	REMEMBER
A timeout is a limit on how long Playwright waits for a particular test, action, navigation, or assertion.	Do not increase every timeout automatically. First identify which timeout is failing and why.
A timeout error is a symptom, not always the root cause.	A misspelled locator, an element that never appears, a slow page, or a real application defect can all lead to waiting.

Learning path

1. Understand the intentionally failing test from class.
2. Learn the difference between test timeout, action timeout, assertion timeout, and navigation timeout.
3. Configure timeout limits at the right scope.
4. Diagnose failures using the call log, locator, DOM, and trace.
5. Apply timeout handling to login, search, checkout, and other real workflows.

Beginner tip: Do not start by memorizing timeout values. Learn what each value controls, what error it produces, and the reason the test is waiting.

Contents

- 1. Timeout vocabulary and mental model
- 2. Class Demo 1: intentionally trigger a timeout
- 3. Read the error and call log
- 4. The main timeout types
- 5. Approach 1: configure the test timeout
- 6. Approach 2: set an action timeout
- 7. Assertion and navigation timeouts
- 8. DOM and locator examples
- 9. Real-time scenarios and worked examples
- 10. Step-by-step debugging checklist
- 11. Common mistakes and best practices
- 12. Practice exercises with answers
- 13. Quick reference and official resources

1. Timeout vocabulary and mental model

A timeout is a maximum amount of time Playwright or the test runner is allowed to wait for an operation or condition. Time is measured in milliseconds (ms). 1,000 ms = 1 second; 30,000 ms = 30 seconds.

Term	Beginner explanation	Typical use
Test timeout	Time budget for an entire test, including its test body and relevant setup.	Stop a test that is stuck or taking too long overall.
Action timeout	Maximum time for a single action such as click(), fill(), check(), or selectOption(), when configured.	Fail quickly when an action cannot complete.
Assertion/expect timeout	How long an auto-retrying assertion waits for the expected result.	Wait for a success message or a loading result to appear.
Navigation timeout	Maximum time allowed for a navigation operation such as page.goto(), when configured.	Limit how long navigation may wait.
Global timeout	Limit for the whole test run, across the suite.	Prevent a full CI job from running indefinitely.
Auto-waiting	Playwright checks whether elements are ready and retries certain operations automatically.	Avoid arbitrary sleeps for ordinary UI readiness.

Timeout values depend on configuration. In the current Playwright Test documentation, the default test timeout is 30 seconds, and the default timeout for auto-retrying expect assertions is 5 seconds. The action timeout and navigation timeout are not limited by a separate timeout by default, but the overall test timeout still applies. Official references are listed at the end.

> **Important distinction:** The test timeout does not mean every action gets 30 seconds independently. It is an overall test budget. Multiple actions share that budget unless action-level timeouts or other settings change the behavior.

2. Class Demo 1: intentionally trigger a timeout

The supplied notes intentionally use the text “Formss”, which is likely a typo for the actual menu label “Forms”. The exact label and role should be confirmed in the current browser DOM.

```
import { test, expect } from '@playwright/test';

test('Handling Timeout Demo 1', async ({ page }) => {
  await page.goto('https://playground.bondaracademy.com/pages/iot-dashboard');

  // Intentionally incorrect locator: extra "s" in Formss.
  await page.getByText('Formss').click();

  // This line will not run if the click above times out.
  await page.waitForTimeout(5000);
});
```

What each line does

- import { test, expect } imports Playwright Test functions. This example does not use expect yet, so the expect import could be removed until an assertion is added.
- test(...) defines one automated test and gives it a readable name.
- async ({ page }) receives the page fixture. The page is the browser tab used by this test.
- page.goto(...) navigates that tab to the dashboard URL.
- page.getByText("Formss") creates a locator for text matching Formss. It does not prove the element exists; the action checks when it tries to click.

- click() tries to locate a matching element and perform a click. If no matching element appears, Playwright keeps waiting until a relevant timeout is reached.
- waitForTimeout(5000) pauses for five seconds, but the test cannot reach this statement if the preceding click has already failed or timed out.

Simplified DOM: why the locator fails

```
<!-- Illustrative DOM only; inspect the live page to confirm its real structure. -->
<nav>
  <a href="/pages/forms">Forms</a>
  <a href="/pages/iot-dashboard">IoT Dashboard</a>
</nav>
```

Locator	Expected outcome	Why
page.getByText('Formss')	No matching node in this illustrative DOM.	The DOM contains “Forms” with one final s, not “Formss”.
page.getByText('Forms')	May match the visible Forms text.	Text matches, but still inspect whether this is the intended element.
page.getByRole('link', { name: 'Forms' })	Matches a link named Forms in this illustrative DOM.	Role and accessible name describe what the element is and how users identify it.

Best fix: Correct the locator first. Raising the timeout cannot make the misspelled text “Formss” appear. A longer timeout would only delay the failure.

3. Read the error and call log

A timeout report usually includes the operation that failed and a call log describing what Playwright was waiting for. The exact wording may differ depending on which timeout is configured and which timeout limit is reached.

```
Example-style error:
  locator.click: Timeout 2000ms exceeded.
  Call log:
    - waiting for getByText('Formss')
```

Read this as: “Playwright attempted to click the locator, but the matching element/action did not become ready within the allowed time.” It is not necessarily saying the browser crashed.

A practical way to read the report

1. Find the first failing line in your spec file.
2. Read the operation name: click(), fill(), goto(), or an expect assertion.
3. Read the locator in the call log. Check spelling, role, accessible name, CSS selector, and whether the intended element exists.
4. Inspect the live DOM with browser DevTools or Playwright Inspector. Search for the target text or attribute.
5. Decide whether the issue is a wrong locator, a slow/late UI state, navigation, an overlay, a disabled control, or a product defect.
6. Only then choose whether to fix the locator, wait for a meaningful condition, or adjust a timeout.

4. The main timeout types

4.1 Test timeout

The test timeout caps the total test body and related setup time. Playwright Test uses 30,000 ms (30 seconds) by default. The test may exceed its budget even if each individual action seems reasonable, because the time is shared across the test.

```
// playwright.config.js / playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 60_000, // 60 seconds for each test
});
```

4.2 Action timeout

An action timeout limits an individual action when configured. For example, a click should not wait too long for a target that never appears. In current Playwright documentation, actionTimeout defaults to 0 (no separate action timeout); the overall test timeout may still stop a test that remains stuck.

```
// playwright.config.js / playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    actionTimeout: 2_000, // 2 seconds per action
  },
});
```

4.3 Assertion timeout

Auto-retrying web assertions such as toBeVisible() and toHaveText() retry until the expected state appears or the assertion timeout expires. The default is 5 seconds. Assertion timeout is separate from the test timeout.

```
await expect(page.getByRole('status'))
  .toHaveText('Order confirmed', { timeout: 10_000 });
```

4.4 Navigation timeout

Navigation timeout controls navigation operations, such as page.goto(). If a site is genuinely slow to navigate, use a deliberate navigation limit rather than increasing every action timeout.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    navigationTimeout: 30_000,
  },
});
```

4.5 Global timeout

A global timeout is an optional cap for the whole run, not just one test. It can be useful in CI to stop a suite that has hung or become unexpectedly slow. Choose it according to the size of your suite and CI job budget.

Timeout setting	Where to set it	What it limits
timeout	Top-level Playwright config	Each test's time budget.

test.setTimeout(...)	Inside a test or an applicable hook	The timeout for the current test.
use.actionTimeout	Inside config use block	Each action's separate timeout.
expect.timeout	Top-level config under expect	Default auto-retrying assertion timeout.
navigationTimeout	Inside config use block	Navigation operations.
globalTimeout	Top-level Playwright config	The complete test run.

5. Approach 1: configure the test timeout

A. Global/framework-level setting

Use this when most tests need a different overall budget. This is configured in playwright.config.js or playwright.config.ts. The setting applies to tests in that configuration, so do not increase it blindly for a single slow test.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 60_000,
  use: {
    trace: 'on-first-retry',
  },
});
```

B. Individual test timeout

Use test.setTimeout() for a specific test that has a legitimate reason to take longer, such as a long multi-step workflow. Put it inside the test body before lengthy operations.

```
import { test, expect } from '@playwright/test';

test('Long report export workflow', async ({ page }) => {
  test.setTimeout(90_000); // 90 seconds for this test only

  await page.goto('https://example.test/reports');
  await page.getByRole('button', { name: 'Generate report' }).click();
  await expect(page.getByRole('status'))
    .toHaveText('Report ready', { timeout: 30_000 });
});
```

Best practice: Increase the timeout only when the workflow legitimately needs more time. For a typo or missing element, fix the test or investigate the app instead.

6. Approach 2: set an action timeout

Your class notes recommend adding actionTimeout: 2000 in the use block. That means each configured Playwright action can fail after approximately two seconds rather than waiting indefinitely for its own action-specific limit. The overall test timeout remains a separate limit.

```
// playwright.config.js / playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 60_000, // overall test budget
  use: {
    actionTimeout: 2_000, // action budget
    trace: 'on-first-retry',
  },
});
```

Think of these as two clocks: the test clock tracks the whole test, while the action clock tracks one operation. A test with a 60-second budget can still fail at the 2-second mark if a click cannot complete within the action timeout.

```
// Override timeout for one particular click
await page.getByRole('button', { name: 'Save' }).click({ timeout: 5_000 });
```

Caution about 2 seconds: Two seconds may be too short for slower CI machines, remote environments, or controls that appear after a network request. Start with sensible project values and observe real run behavior. Avoid making the entire suite flaky by choosing an unrealistically low limit.

7. Assertion and navigation timeouts

7.1 Wait for a meaningful UI condition

```
// After submitting a registration form
await page.getByRole('button', { name: 'Register' }).click();
await expect(page.getByRole('status'))
  .toHaveText('Registration successful');
```

The assertion retries while the status element is not yet showing the expected text. It is stronger than sleeping for an arbitrary amount of time because it checks the actual outcome you care about.

7.2 Override one assertion only

```
await expect(page.getByRole('heading', { name: 'Monthly report' }))
  .toBeVisible({ timeout: 15_000 });
```

Use a longer assertion timeout when the expected state legitimately takes longer, for example a report generated by a backend job. If it never appears, the test still fails and reports the unmet condition.

7.3 Avoid fixed sleeps for synchronization

```
// Fragile: guesses that the UI will be ready after 5 seconds
await page.waitForTimeout(5_000);

// Better: wait for the actual state
await expect(page.getByRole('status')).toHaveText('Saved');
```

page.waitForTimeout() is discouraged for production test synchronization because fixed delays can be too short on a slow run and waste time on a fast run. It can still be useful temporarily while debugging, but prefer locator actions, web-first assertions, or a specific event/condition.

8. DOM and locator examples

A timeout can be caused by a locator that does not correspond to the DOM. A DOM (Document Object Model) is the browser's tree-like representation of the page HTML. Before changing timeouts, make sure you are locating a real element correctly.

Example A: misspelled menu item

```
<!-- Illustrative DOM -->
<nav>
  <a href="/forms">Forms</a>
</nav>

// Wrong
await page.getByText('Formss').click();

// Better if it is a link
await page.getByRole('link', { name: 'Forms', exact: true }).click();
```

Example B: button appears after submitting a form

```
<!-- Simplified DOM before submit -->
<form>
  <input id="email" type="email" />
  <button type="submit">Continue</button>
</form>
<div role="status" aria-live="polite"></div>

<!-- After successful response, the status may become: -->
<div role="status" aria-live="polite">Verification email sent</div>
```

```
await page.getByLabel('Email').fill('qa@example.test');
await page.getByRole('button', { name: 'Continue' }).click();
await expect(page.getByRole('status'))
  .toHaveText('Verification email sent', { timeout: 10_000 });
```

Why this is better: the test validates the result, not just that a click was attempted. If the application never sends the status message, the failed assertion gives a meaningful clue.

Example C: a disabled or covered button

```
<!-- Example states -->
<button disabled>Save</button>
<!-- or an overlay blocks the button -->
<div class="loading-overlay">Saving...</div>
<button>Save</button>
```

Playwright may wait because the element is not actionable yet. Inspect the call log. Confirm the control becomes enabled, an overlay disappears, and the intended button is unique. Do not use force: true as a reflex; that can hide real UI problems.

9. Real-time scenarios and worked examples

Scenario 1: Login takes time

Business situation: after the user clicks Sign in, the app validates credentials and redirects to a dashboard. The test should wait for the dashboard heading, not sleep for a fixed number of seconds.

```
import { test, expect } from '@playwright/test';

test('User can sign in', async ({ page }) => {
  await page.goto('https://example.test/login');
  await page.getByLabel('Email').fill('qa.user@example.test');
  await page.getByLabel('Password').fill('test-only-password');
  await page.getByRole('button', { name: 'Sign in' }).click();

  await expect(page.getByRole('heading', { name: 'Dashboard' }))
    .toBeVisible({ timeout: 10_000 });
});
```

If the timeout happens, check: did login succeed, is the expected heading text correct, did the app show an error banner, or did the backend response fail? A longer timeout does not repair invalid credentials.

Scenario 2: Search results load asynchronously

```
await page.getByRole('searchbox', { name: 'Search products' }).fill('laptop');
await page.getByRole('button', { name: 'Search' }).click();
await expect(page.getByRole('heading', { name: /search results/i }))
  .toBeVisible({ timeout: 10_000 });
await expect(page.getByTestId('product-card').first()).toBeVisible();
```

Use robust accessible names or stable test IDs that reflect the actual application DOM. If the page uses a different name, inspect it and update the sample locator. An assertion should reflect what success means for this feature.

Scenario 3: Slow report generation

```
test('Generate monthly report', async ({ page }) => {
  test.setTimeout(120_000); // a justified longer end-to-end test budget

  await page.goto('https://example.test/reports');
  await page.getByRole('button', { name: 'Generate monthly report' }).click();
  await expect(page.getByRole('status'))
    .toHaveText('Report generated', { timeout: 60_000 });
});
```

A background report may legitimately take time, so both the total test budget and the assertion budget are chosen deliberately. In a real system, consider checking a job status or API response if appropriate, instead of sleeping.

Scenario 4: Checkout total never updates

```
await page.getByRole('button', { name: 'Apply coupon' }).click();
await expect(page.getByTestId('order-total')).toHaveText('$90.00', {
  timeout: 8_000,
});
```

If the total stays unchanged, the cause may be an invalid coupon, a frontend bug, a backend failure, or a wrong expected amount. Debug the business flow and test data before increasing timeouts.

Scenario 5: Navigation to a remote page is slow

```
await page.goto('https://example.test/slow-page', {
  timeout: 30_000,
  waitUntil: 'domcontentloaded',
});
```

The waitUntil option should match the page behavior you need. domcontentloaded means the initial HTML document has been parsed; it does not guarantee every image or asynchronous widget is ready. Follow it with a locator assertion for the specific control needed by the test.

10. Step-by-step debugging checklist

1. Re-run the test and identify the exact operation that timed out: navigation, action, assertion, or whole test.
2. Read the call log and note the locator or condition being waited for.
3. Inspect the current page DOM and verify the element really exists. Check spelling, casing, attributes, role, and accessible name.
4. Confirm the element is not inside an iframe, popup, or another page context. Use the correct page/frame locator if applicable.
5. Check whether an overlay, animation, disabled state, network request, or app error prevents the operation.
6. Replace fixed sleeps with an assertion or explicit condition that represents success.
7. Set the smallest appropriate timeout at the right level; do not change all timeout settings just to silence one failure.
8. Use the Playwright HTML report and trace when available. In config, trace: "on-first-retry" is a practical starting point for diagnosis.
9. Run the test again and ensure it is stable, not merely passing once.

11. Common mistakes and best practices

Mistake	Why it is risky	Better approach
Increasing the test timeout for a typo	It makes the failure slower but does not fix the cause.	Verify the locator against the DOM first.
Using waitForTimeout(5000) after every action	Adds delays and remains unreliable under variable load.	Use web-first assertions such as toBeVisible() or toHaveText().
Setting actionTimeout to 2 seconds everywhere without testing	A legitimate slow CI action may fail.	Pick measured, realistic limits for your project and environment.
Treating all timeout errors as the same	Test, action, navigation, and assertion timeouts have different scopes.	Read the operation and timeout text in the report.
Using force: true to get past a click	Can hide overlays, disabled controls, or a genuine usability bug.	Find out why the element is not actionable.
Not awaiting an assertion	The asynchronous assertion may not be completed correctly.	Use await expect(locator).toBeVisible() etc.
Expecting a button click alone to prove success	A click can occur without the intended business result.	Assert the resulting UI state or application response.
Globally making every test very long	Failures consume more CI time and slow feedback.	Scope longer limits to tests that need them.

12. Practice exercises with answers

Exercise 1: identify the error

```
await page.getByText('Formss').click();
```

Question: what should you check before increasing the timeout?

Answer: inspect the DOM and confirm the exact rendered text, role, and uniqueness. In the class example, Formss is intentionally incorrect and may need to be Forms. Then choose a locator that matches the actual element.

Exercise 2: set test timeout to 60 seconds

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  timeout: 60_000,
});
```

Explanation: this changes the overall timeout per test at configuration level. It does not specifically set every click to 60 seconds.

Exercise 3: set a two-second action timeout

```
export default defineConfig({
  use: {
    actionTimeout: 2_000,
  },
});
```

Explanation: actions that do not complete within the configured action budget fail faster. Select this value only if it suits your environment.

Exercise 4: replace a fixed wait

```
// Instead of this:
await page.waitForTimeout(5000);


// Use an assertion tied to the result:
await expect(page.getByRole('status')).toHaveText('Saved');
```

Explanation: the assertion waits for the state being tested instead of assuming that five seconds is enough.

Exercise 5: distinguish failure types

Symptom	Likely timeout family to inspect first
page.goto() does not finish	Navigation timeout and navigation wait condition
button click waits for element	Action timeout, locator correctness, actionability
success text does not appear	Assertion timeout and application outcome
whole test takes too long	Test timeout and overall workflow efficiency
the entire suite runs indefinitely	Optional global timeout and hanging workers/tests

13. Quick reference and official resources

Need to do	Example
Set global/per-test budget	timeout: 60_000
Set this test's budget	test.setTimeout(60_000)
Set an action budget	use: { actionTimeout: 2_000 }
Set expect default	expect: { timeout: 10_000 }
Override one assertion	await expect(locator).toBeVisible({ timeout: 10_000 })
Set navigation budget	use: { navigationTimeout: 30_000 }
Inspect a failing action	Read locator.click and the call log in the report
Prefer event/state waits	await expect(locator).toHaveText("Saved")

Official documentation (consulted 29 October 2026):

- Timeouts: https://playwright.dev/docs/test-timeouts
- Assertions: https://playwright.dev/docs/test-assertions
- Configuration and use options: https://playwright.dev/docs/test-use-options
- Page API / waitForTimeout guidance: https://playwright.dev/docs/api/class-page

Final lesson: When Playwright reports a timeout, ask: what exactly was it waiting for, what DOM/state did I expect, and which timeout controls that operation? Correct the root cause first; adjust the timeout only when the operation genuinely needs more time.